41039 Programming 1

Revision of Week 7: Lists and generics



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the revision session works

A recap before the next lecture

Write an answer before the reveal. Explain the state change, rule or contract that makes it follow. These are practice questions, with no new assessment requirements.

Part 1: on your own

- 6 short questions
- About 90 seconds per question
- Java and Python; name the language first

Part 2: teams of 3 or 4

- Draw the model
- Spot the errors
- Predict the output
- Write and test the code

A result needs a reason. A repair needs a counterexample.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

1. Changing a resizable list

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.util.ArrayList;
ArrayList<String> names = new ArrayList<String>();
names.add("An");
names.add("Mai");
names.add(1,"Linh");
names.set(0,"Binh");
System.out.println(names);
System.out.println(names.size());
```

Your answer

What is the complete output? Which line or rule decides it?

1. Changing a resizable list

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.util.ArrayList;
ArrayList<String> names = new ArrayList<String>();
names.add("An");
names.add("Mai");
names.add(1,"Linh");
names.set(0,"Binh");
System.out.println(names);
System.out.println(names.size());
```

Answer

```
[Binh, Linh, Mai]
3
```

add at an index inserts; set replaces. Three elements remain.

2. The comparable Python operations

Python 3; commit to an outcome before the reveal.

```
names = ["An", "Mai"]
names.insert(1, "Linh")
names[0] = "Binh"
print(names)
print(len(names))
```

Your answer

What is the complete output? Which line or rule decides it?

2. The comparable Python operations

Python 3; commit to an outcome before the reveal.

```
names = ["An", "Mai"]
names.insert(1, "Linh")
names[0] = "Binh"
print(names)
print(len(names))
```

Answer

```
['Binh', 'Linh', 'Mai']
3
```

Python lists offer the same insert/replace distinction, with different syntax.

3. Remove an index or remove a value?

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.util.ArrayList;
import java.util.Arrays;
ArrayList<Integer> values = new ArrayList<Integer>(
    Arrays.asList(4,1,4));
values.remove(1);
values.remove(Integer.valueOf(4));
System.out.println(values);
```

Your answer

What is the complete output? Which line or rule decides it?

3. Remove an index or remove a value?

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
import java.util.ArrayList;
import java.util.Arrays;
ArrayList<Integer> values = new ArrayList<Integer>(
    Arrays.asList(4,1,4));
values.remove(1);
values.remove(Integer.valueOf(4));
System.out.println(values);
```

Answer

[4]

The index removal leaves [4,4]; the value removal deletes only the first 4.

4. Safe filtering produces a new list

Python 3; commit to an outcome before the reveal.

```
values = [1, 2, 2, 3, 4]
kept = [value for value in values if value % 2 == 1]
print(kept)
print(values)
```

Your answer

What is the complete output? Which line or rule decides it?

4. Safe filtering produces a new list

Python 3; commit to an outcome before the reveal.

```
values = [1, 2, 2, 3, 4]
kept = [value for value in values if value % 2 == 1]
print(kept)
print(values)
```

Answer

```
[1, 3]
[1, 2, 2, 3, 4]
```

Filtering leaves the source unchanged and keeps both order and only the matching values.

5. A reusable, type-safe box

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Box<String> label = new Box<String>("tea");
Box<Integer> count = new Box<Integer>(3);
System.out.println(label.get() + " " + count.get());
class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}
```

Your answer

What is the complete output? Which line or rule decides it?

5. A reusable, type-safe box

Java 8 excerpt: main statements and supporting types; commit to an outcome before the reveal.

```
Box<String> label = new Box<String>("tea");
Box<Integer> count = new Box<Integer>(3);
System.out.println(label.get() + " " + count.get());
class Box<T> {
    private final T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
}
```

Answer

```
tea 3
```

The class is reused with two types while each individual box retains a checked element type.

6. The same FIFO contract with deque

Python 3; commit to an outcome before the reveal.

```
from collections import deque
queue = deque(["An", "Mai"])
print(queue.popleft())
print(list(queue))
```

Your answer

Predict the output. Which Java list provides addLast/removeFirst? Why would repeated get(i) on that list be expensive?

6. The same FIFO contract with deque

Python 3; commit to an outcome before the reveal.

```
from collections import deque
queue = deque(["An", "Mai"])
print(queue.popleft())
print(list(queue))
```

Answer

```
An
['Mai']
```

popleft removes An, and Mai remains waiting. Java LinkedList provides addLast/removeFirst. Finding an arbitrary index requires traversing links, so repeatedly accessing every index can take quadratic time. Prefer sequential iteration for that access pattern.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Form teams

- Teams of 3 or 4. One shared sheet, one spokesperson.
- For each challenge, record the result and the reason.
- Rotate the spokesperson. Other teams can challenge an explanation.
- The final task has starter files and a Java/Python solution pair.

Discuss first. Commit to an answer. Then compare with the reveal.

Draw it: A list copy and a reference copy

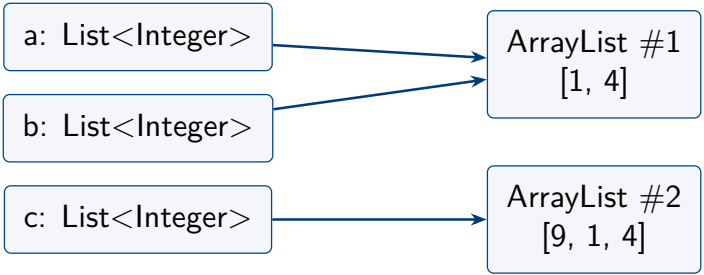
```
import java.util.*;
List<Integer> a=new ArrayList<Integer>(Arrays.asList
    (4,1,4));
List<Integer> b=a;
List<Integer> c=new ArrayList<Integer>(a);
b.remove(Integer.valueOf(4));
c.set(0,9);
System.out.println(a);
System.out.println(b);
System.out.println(c);
```

Course website: Week 7. Complete programs accompany this deck.

Four minutes

Draw the object references after the shown operations. Label the important state. Mark what is shared, what changes, or which path handles failure. Predict every output line.

Draw it: one answer



Output

```
[1, 4]
[1, 4]
[9, 1, 4]
```

new ArrayList(a) creates a distinct list container. The elements here are immutable Integer values, so a shallow copy is sufficient for independent changes to list positions.

Course website: Week 7. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Spot the errors 1: Java (5 min)

Contract

Remove one occurrence of target by value, then every 9. Expected final list: [2]. Test target values that differ from their positions.

```java
import java.util.*;
public class Broken {
    public static void main(String[] args) {
        ArrayList<Integer> xs=new ArrayList<Integer>(Arrays.asList(9,1,9,2));
        int target=1;
        xs.remove(target);
        for(int i=0;i<xs.size();i++) {
            if(xs.get(i)==9) xs.remove(i);
        }
        System.out.println(xs);
    }
}
```

Spot the errors 1: answers

The repairs

- **L6:** An int selects index removal; wrap target to select removal by value. This data initially hides the mistake.
- **L7:** Removing while increasing i skips an adjacent match after elements shift. Traverse indices backwards.

Course website: Week 7. Complete programs accompany this deck.

✗ Original program

```
[9, 2]
```

✓ After the repairs

```
[2]
```

Spot the errors 2: Python (5 min)

Contract

Keep odd values in a new list, preserving order and leaving values unchanged.

```
values=[2,2,3,4,4,5]
kept=values
for value in kept:
    if value%2==0: kept.remove(value)
print(kept)
print(values)
```

Spot the errors 2: answers

The repairs

- **L2:** Keep the original list unchanged by building a new result.
- **L3:** Removing from the list being traversed skips adjacent even values. Filter into a separate list.

Course website: Week 7. Complete programs accompany this deck.

✗ Original program

```
[2, 3, 4, 5]
[2, 3, 4, 5]
```

✓ After the repairs

```
[3, 5]
[2, 2, 3, 4, 4, 5]
```

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Predict 1: Three operations, two overloads

```
import java.util.*;
List<Integer> xs=new ArrayList<Integer>(Arrays.asList
    (1,2,1));
xs.remove(1);
xs.add(1,4);
System.out.println(xs);
xs.remove(Integer.valueOf(1));
System.out.println(xs);
System.out.println(xs.size());
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 7. Complete programs accompany this deck.

Predict 1: answer

```
[1, 4, 1]
[4, 1]
2
```

Why

The first remove selects an index. add inserts without overwriting, then the Integer argument selects first-value removal.

Course website: Week 7. Complete programs accompany this deck.

Predict 2: Queue order after mixed operations

```
from collections import deque
q=deque(["An","Mai"])
q.append("Linh")
served=q.popleft()
q.appendleft("Tea")
print(served)
print(list(q))
print(len(q))
```

Three minutes

Write every line in order. Explain the first state change, alias, dispatch decision or exception that controls the result. Then propose one change that alters the output.

Course website: Week 7. Complete programs accompany this deck.

Predict 2: answer

```
An
['Tea', 'Mai', 'Linh']
3
```

Why

popleft serves the oldest entry. appendleft then puts Tea before the remaining names.

Course website: Week 7. Complete programs accompany this deck.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

Code it: specification

First occurrences, in order

Read n and n integers. Print each distinct value once in first-occurrence order, separated by single spaces. Empty input produces an empty line. Explain why sorting changes the contract.

Example input

```
5
4 1 4 2 1
```

Expected output

```
4 1 2
```

Eight minutes. One Java team and one Python team at the keyboard. Other teams produce a boundary test and explain which wrong implementation it would expose.

Code it: Java solution

```java
import java.util.*;
Scanner sc=new Scanner(System.in);
int n=sc.nextInt();
List<Integer> result=new ArrayList<Integer>();
for(int i=0;i<n;i++) {
    int x=sc.nextInt();
    if(!result.contains(x)) result.add(x);
}
for(int i=0;i<result.size();i++) {
    if(i>0) System.out.print(" ");
    System.out.print(result.get(i));
}
System.out.println();
```

Course website: Week 7. Complete programs accompany this deck.

Code it: Python solution

```python
n=int(input())
values=list(map(int,input().split()))
result=[]
for x in values[:n]:
    if x not in result: result.append(x)
print(*result)
```

Course website: Week 7. Complete programs accompany this deck.

Code it: the tests

A slash separates input lines; (blank) denotes an empty line.

Test 1
`5 / 4 1 4 2 1`
Output: `4 1 2`

Test 2
`0 / (blank)`
Output: `(empty line)`

Test 3
`4 / 7 7 7 7`
Output: `7`

Test 4
`3 / -1 0 -1`
Output: `-1 0`

Explain why each test is present, not only whether it passes.

Outline

1. On your own
2. Team round 1: draw the model
3. Team round 2: spot the errors
4. Team round 3: predict the output
5. Team round 4: write the code
6. Wrapping up

What you should now be able to explain

- Distinguish append, insert, replace and both remove overloads.
- Explain aliasing versus a copied list container.
- Choose ArrayList, LinkedList or deque from the required operations.
- Use wrapper types and type parameters without discarding type safety.

Return to the matching week page for runnable examples and further practice. Bring one unresolved prediction or counterexample to the next class.

Course website: Week 7. Complete programs accompany this deck.